

TÜRKİYE CUMHURİYETİ
YILDIZ TEKNİK ÜNİVERSİTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



OTONOM AJAN YAKLAŞIMIYLA KOD VERİ
TABANINDAN EĞİTİM İÇERİĞİ ÜRETEEN YAPAY ZEKA
TABANLI YAZILIMIN TASARIMI VE GELİŞTİRMESİ

21011035 – BORA İLCİ
21011506 – KAAN EMRE KARA

BİLGİSAYAR PROJESİ

Danışman
Prof. Dr. Mehmet Sıddık AKTAŞ

Ocak, 2026

TEŞEKKÜR

Yıldız Teknik Üniversitesi, tarihi 1911'e dayanan Türkiye'nin en eski mühendislik okullarından biri olmasının yanı sıra, ülkemizin en prestijli eğitim kurumlarından kabul edilmektedir. Üniversitemiz bugün 10 Fakültesi, 2 Enstitüsü ve 25.000'den fazla öğrencisiyle bilim ve teknoloji dünyasına katkı sunmaya devam etmektedir.

Kurumumuz, 20 Haziran 1982 tarih ve 41 sayılı kanun hükmünde kararname ile Yıldız Üniversitesi adını almış; Fen-Edebiyat ve Mühendislik bölümleriyle akademik altyapısını güçlendirmiştir. Günümüzde geniş kampüs olanakları ve güçlü akademik kadrosuyla, özellikle mühendislik ve mimarlık başta olmak üzere pek çok alanda öncü bir rol oynamaktadır. Bu projenin gerçekleştirilmesinde emeği geçen tüm kurumsal ve bireysel paydaşlara teşekkür ederiz.

BORA İLCİ
KAAN EMRE KARA

İÇİNDEKİLER

KISALTMA LİSTESİ	v
ŞEKİL LİSTESİ	vi
TABLO LİSTESİ	vii
ÖZET	viii
ABSTRACT	ix
1 GİRİŞ	1
2 SİSTEM ANALİZİ VE FİZİBİLİTE	5
2.1 Sistem Analizi	5
2.2 Gereksinim Analizi	7
2.3 Fizibilite	8
2.3.1 Teknik Fizibilite	8
2.3.2 Yasal Fizibilite	8
2.3.3 Ekonomik Fizibilite	8
2.3.4 İşgücü ve Zaman Fizibilitesi	9
3 SİSTEM TASARIMI	11
3.1 Arayüz Tasarımı	11
3.1.1 Komut Satırı Arayüzü (CLI)	11
3.1.2 Web Tabanlı Kontrol Paneli (Web Dashboard)	11
3.2 Veri Yapısı Tasarımı	12
3.2.1 Yapılandırma Yönetimi	12
3.2.2 İzole Çalışma Alanları (Scoped Filesystem)	12
3.3 Nesneye Yönelik Tasarım ve Ajan Mimarisi	13
3.3.1 Dinamik Ajan Yapısı	13
3.3.2 Orkestrasyon Katmanı	14
4 UYGULAMA SONUÇLARI ve TESTLER	15
4.1 Uygulama Tanıtımı	15

4.2	Test Sonuçları	15
4.2.1	Maliyet ve Bağlam Analizi (Deney 4)	16
4.3	Doğrulama	16
4.3.1	Yargıç LLM Değerlendirmesi (Deney 2)	17
4.3.2	Gereksinim Uyumluluk Analizi	17
5	SONUÇ VE TARTIŞMA	19
5.1	Elde Edilen Kazanımlar	19
5.2	Kısıtlar ve Maliyet	20
5.3	Gelecek Çalışmalar	20
	Referanslar	21
	Özgeçmiş	21

KISALTMA LİSTESİ

LLM Large Language Model (Büyük Dil Modeli)

RAG Retrieval-Augmented Generation (Geri Getirme Artırılmış Üretim)

API Application Programming Interface (Uygulama Programlama Arayüzü)

CLI Command Line Interface (Komut Satırı Arayüzü)

KB Knowledge Base (Bilgi Bankası)

SDLC Software Development Life Cycle (Yazılım Geliştirme Yaşam Döngüsü)

MIT Massachusetts Institute of Technology License

ŞEKİL LİSTESİ

Şekil 2.1	Bilgi üretim hattının üst düzey görünümü.	5
Şekil 2.2	İşgücü Fizibilitesi – Rol Bazlı Gantt Şeması	10
Şekil 3.1	Kullanıcı etkileşimi ve veri akış şeması.	12
Şekil 3.2	Orkestrasyon yapısı ve dinamik ajan üretimi.	14

TABLO LİSTESİ

Tablo 4.1	Baseline ve Deep Agent Modellerinin Tutorial Üretim Fazı Karşılaştırması	16
Tablo 4.2	Yargıç LLM'lerin Kalite Değerlendirmesi (5 üzerinden puan) . . .	17

OTONOM AJAN YAKLAŞIMIYLA KOD VERİ TABANINDAN EĞİTİM İÇERİĞİ ÜRETEEN YAPAY ZEKA TABANLI YAZILIMIN TASARIMI VE GELİŞTİRMESİ

BORA İLCİ
KAAN EMRE KARA

Bilgisayar Mühendisliği Bölümü
Bilgisayar Projesi

Danışman: Prof. Dr. Mehmet Sıddık AKTAŞ

Bu proje, büyük ve karmaşık kod tabanlarını anlama ve dokümantasyon oluşturma zorluğunu, yüksek kaliteli eğitim içeriği üretebilen otonom bir yapay zeka ajanı geliştirerek çözmeyi amaçlamaktadır. Geleneksel tek ajanlı yaklaşımlar, Büyük Dil Modellerinin (LLM) sınırlı bağlam penceresi nedeniyle geniş kod depolarını işlerken genellikle başarısız olmaktadır. Bu kısıtlamayı aşmak için, *Sub-Agent* metodolojisine dayalı özgün bir *Deep Agent* mimarisi geliştirilmiştir.

Sistem iki aşamada çalışmaktadır: İlk aşamada, bir Supervisor Agent, kod tabanının farklı bölümlerini izole olarak analiz etmek için birden fazla Sub-Agent görevlendirir ve bağlamı aşırı yüklemekten yapılandırılmış bir Knowledge Base oluşturur. İkinci aşamada, bir Tutorial Generator bu bilgi bankasını kullanarak kapsamlı, adım adım ilerleyen eğitici markdown dosyaları üretir. Önerilen sistem, "Agent Lightning" gibi karmaşık kütüphaneler üzerinde test edilmiş ve bir Temel (Baseline) yaklaşımla karşılaştırılmıştır. Yargıç LLM deneylerinden (Claude 4.5 Sonnet, GPT-OSS-120B, Gemini 2.5 Pro ve Grok Code Fast 1 kullanılarak) elde edilen sonuçlar, Deep Agent mimarisinin doğruluk, pedagoji ve kapsam açısından temel yaklaşımdan önemli ölçüde üstün olduğunu ve ham kodu değerli öğrenme kaynaklarına dönüştürmede etkili olduğunu göstermektedir.

Anahtar Kelimeler: Deep Agent, Multi-Agent Systems, Otonom Dokümantasyon, Büyük Dil Modelleri, Bağlam Yönetimi, Yazılım Mühendisliği Eğitimi

ABSTRACT

DESIGN AND DEVELOPMENT OF AN ARTIFICIAL INTELLIGENCE-BASED SOFTWARE THAT GENERATES EDUCATIONAL CONTENT FROM A CODE DATABASE USING AN AUTONOMOUS AGENT APPROACH

BORA İLCİ
KAAN EMRE KARA

Department of Computer Engineering
Computer Project

Advisor: Prof. Dr. Mehmet Sıddık AKTAŞ

This project addresses the challenge of understanding and documenting large, unfamiliar codebases by developing an autonomous artificial intelligence agent capable of generating high-quality educational content. Traditional single-agent approaches often fail when processing large repositories due to the finite context window of Large Language Models (LLMs). To overcome this limitation, a novel *Deep Agent* architecture based on the *Sub-Agent* methodology has been implemented.

The system operates in two phases: first, a Supervisor Agent orchestrates multiple Sub-Agents to analyze isolated parts of the codebase, generating a structured Knowledge Base without overloading the context. Second, a Tutorial Generator utilizes this knowledge base to produce comprehensive, step-by-step educational markdown files. The proposed system was tested on complex libraries such as "Agent Lightning" and evaluated against a Baseline (single-agent) approach. Results from LLM-as-a-Judge experiments (using Claude 4.5 Sonnet, GPT-OSS-120B, Gemini 2.5 Pro, and Grok Code Fast 1) demonstrate that the Deep Agent architecture significantly outperforms the baseline in terms of fidelity, pedagogy, and coverage, effectively transforming raw code into valuable learning resources.

Keywords: Deep Agents, Multi-Agent Systems, Automated Documentation, Large Language Models, Context Management, Software Engineering Education

1 GİRİŞ

Modern yapay zeka uygulamalarının temelinde, devasa metin ve kod veri kümeleri üzerinde eğitilmiş Büyük Dil Modelleri (LLM) yer almaktadır. Bu modeller, kendilerine sunulan bilgiyi anlama ve üretme konusunda yüksek kapasiteye sahip olsalar da, tek başlarına karmaşık ve çok aşamalı görevleri yerine getirmede sınırlı kalmaktadır. Bu noktada ajan tabanlı sistemler devreye girmektedir. Bir ajan, LLM'yi karar alma çekirdeği olarak kullanır; hedefe yönelik planlama yapar, dosya okuma veya kod yürütme gibi yardımcı araçları etkinleştirir ve çevreden aldığı geri bildirimle göre stratejisini günceller.

Örneğin finans teknolojileri alanında çalışan bir yazılım mühendisinin, on binlerce dosyadan oluşan yeni bir ödeme altyapısına katıldığını düşünelim. Geliştiricinin mimariyi anlaması, iş akışlarını kavraması ve basit bir değişikliği güvenle uygulayabilmesi çoğu zaman haftalar almaktadır. İnsan gözetiminde yürüyen bu uzun uyum sağlama süreci, kod tabanından yapılandırılmış bilgi çıkarabilen otonom sistemlerin yokluğunda daha da uzamakta; iş devri ve bakım maliyetleri yükselmektedir.

Bu mimarilerin en belirgin teknik kısıtlaması, LLM'lerin "bağlam penceresi" olarak bilinen veri işleme sınırıdır. Bir model aynı anda yalnızca sınırlı miktarda bilgiyi hafızasında tutabilir. Dolayısıyla kapsamlı bir yazılım projesinin tamamını tek seferde değerlendirmek geleneksel ajan yaklaşımlarıyla mümkün olamamaktadır.

Ajan tabanlı sistemler, yazılım geliştirme yaşam döngüsünün (SDLC) birçok aşamasını otomatize etme potansiyeline sahiptir. Bu sistemler, basit kod tamamlama araçlarının çok ötesine geçerek aktif roller üstlenmeye başlamıştır. Güncel uygulama alanları arasında şunlar bulunmaktadır:

- **Otomatik Hata Ayıklama (Debugging):** Bir kod tabanındaki hataları tespit etmek için test senaryoları yürütebilen, günlük (log) dosyalarını analiz edebilen ve hatanın kaynağını belirleyebilen ajanlar.

- **Talep Üzerine Kod Üretimi:** "Kullanıcılar için bir giriş (login) sayfası oluştur" gibi doğal dil taleplerini alıp, gerekli dosya yapılarını ve kod bloklarını üreten sistemler.
- **Mevcut Kodun Yeniden Yapılandırılması (Refactoring):** Verimsiz veya eski kod standartlarına göre yazılmış bölümleri analiz edip, daha performanslı veya modern eşdeğerleriyle güncelleyen ajanlar.
- **Kod Analizi ve Dokümantasyon:** Bu projenin de odak noktası olan, mevcut bir kod tabanını inceleyerek sistemin mimarisini, veri akışlarını ve bileşenlerin sorumluluklarını anlatan teknik dokümantasyon üreten sistemler.

Yazılım geliştirme süreçlerinde, geniş ve yabancı bir kod tabanını anlamak, yazılım geliştiricilerin karşılaştığı en önemli zorluklardan biridir. Yeni bir geliştiricinin bir projeye dahil olması veya tecrübeli bir geliştiricinin projenin daha önce dokunmadığı bir bölümüne müdahale etmesi gerektiğinde, mevcut dokümantasyon genellikle eksik, güncel olmayan veya yüzeysel kalmaktadır. Bu durum, projelere adaptasyon sürecinin haftalar, hatta aylar sürmesine neden olarak ciddi bir zaman ve verimlilik kaybına yol açmaktadır.

Tıpkı geniş ölçekli çevrim içi platformlarda olduğu gibi, modern bir kurumsal kod tabanı binlerce dosya, on binlerce işlev ve milyonlarca satır kod içerebilir. Bu büyüklükteki ve yüksek derecede bağlı verinin, tek bir kişinin kısa sürede incelemesi ve anlamlandırması mümkün değildir. Son yıllarda bu sorunu aşmak için büyük dil modellerinden yararlanan ajan tabanlı yapay zekâ çözümleri geliştirilmektedir. Ancak söz konusu çözümler bile yüksek hacimli kod yığını bütünüyle kavramada bağlam penceresi sınırlamasına takılmaktadır.

Bu sınırlamanın temel nedeni, LLM'lerin belirli bir bağlam uzunluğunu aşamaması ve bu nedenle kod tabanındaki anlamsal bütünlüğü tek seferde kuramamasıdır. Kod tabanını bütünüyle modele yüklemek ya da aynı ajanın tüm dosyaları art arda okumasını beklemek, bağlamın dağılmasına ve yapılan değerlendirmelerin güvenilirliğinin azalmasına yol açmaktadır.

Bu çalışma kapsamında, bağlam penceresi kısıtlamasını aşmak için Sub-Agent mimarisi adını verdiğimiz özgün bir yöntem benimsenmiştir. Söz konusu yaklaşımda kapsamlı analiz görevi daha küçük ve yönetilebilir alt görevlere bölünmektedir.

Projenin temel hedefi, söz konusu mimariyi kullanarak bir "yapay zekâ eğitmeni" prototipi geliştirmektir. Sistem iki ana aşamada işlemektedir:

1. **Aşama 1: Knowledge Base Üretimi:** Supervisor Agent, analiz edilecek yazılım projesini mantıksal modüllerine ayırır. Her bir bölümün analizi için, o bölüme özel, izole bir çalışma alanına ve kısıtlı araçlara sahip bir Sub-Agent görevlendirir. Sub-Agent kendi analizini tamamlar ve bulgularını (markdown dosyaları, diyagramlar) belirlenen dizin altında merkezi bir Knowledge Base oluşturacak şekilde kaydeder. Bu yaklaşım, Supervisor Agent'ın bağlamını temiz tutar ve görevi ölçeklenebilir hale getirir.
2. **Aşama 2: Eğitim Dokümanı Üretimi:** İlk aşamada üretilen ve kod tabanının bütününe özetleyen Knowledge Base bu bölümde girdi olarak kullanılır. Tutorial Generator bileşeni, söz konusu bilgi birikimini ve doğrudan kaynak koddan alınan güncel örnekleri bir araya getirerek geliştiricilere yönelik öğretici dokümanlar üretir.

Bu projenin seçilmesinin temel gerekçesi, özellikle üniversite öğrencileri ve stajyer geliştiriciler için kaynak hazırlanırken karşılaşılan bilgi derleme güçlüğüne ortadan kaldırmaktır. Klasik dokümantasyon süreçleri uzun zaman aldığından pek çok kurum güncel eğitim materyali sunamamaktadır; Sub-Agent destekli yarı otonom bir çözüm bu boşluğu kapatmayı amaçlamaktadır.

Çalışmanın özgün katkıları aşağıdaki başlıklar altında toplanabilir:

- **Kararlı görev bölme yaklaşımı:** Supervisor Agent'ın kod tabanının dizin yapısını inceleyerek ilk seviye klasörlere göre tutarlı alt görevler oluşturması sağlanmış, böylece karmaşık planlama adımlarının yalın bir kural ile yürütülmesi mümkün kılınmıştır.
- **İzole çalışma alanları:** Her bir Sub-Agent'ın yalnızca kendi çalışma dizinine yazabildiği, kod tabanına ise salt okunur eriştiği güvenli bir görev yürütme modeli tasarlanmış, bağlam taşması ve veri bütünlüğü riskleri azaltılmıştır.
- **İki aşamalı üretim hattı:** Knowledge Base ile başlayan ve eğitim dokümanı üretimiyle tamamlanan süreç, dokümantasyon kalitesini yükseltirken ortaya çıkan içeriğin doğrudan öğretim amaçlı kullanılmasına olanak tanımaktadır.

Yazılım endüstrisinde geniş ve yabancı kod tabanlarını anlamak, dokümantasyon eksiklikleri nedeniyle ciddi bir darboğaz oluşturmaktadır. Yeni geliştiricilerin çok sayıda dosyayı tek tek incelemesi yüksek zaman maliyetine yol açmakta; yorumlama süreci ise kişinin bilgi düzeyine bağlı olarak tutarsız sonuçlar üretebilmektedir.

Bu alanda kullanılan ilk yöntemler, AutoGPT benzeri tek ajanlı kod inceleme sistemleridir. Bu sistemler araç kullanımıyla belirli görevleri yerine getirebilse de, LLM bağlam penceresi sınırlaması nedeniyle çok sayıda dosyayı aynı oturumda tutarlı biçimde analiz edememiş; uzun görevlerde odak kaybı ve hatalı çıkarımlar gözlenmiştir.

Sonraki aşamada ortaya çıkan yönetici-işçi düzenindeki çoklu ajan mimarileri, LangChain tabanlı örneklerde olduğu gibi görev dağılımı yaparak kısmi iyileştirme sağlamıştır. Ancak işçilerden gelen çıktının tekrar yöneticide birikmesi bağlam taşmasına, görevlerin yöneticinin belirlediği sıraya sıkı sıkıya bağlı olması ise esneklik kaybına neden olmuştur. Bu yapı yüksek hacimli depo analizlerinde ölçeklenebilirlik sorunlarını ortadan kaldıramamıştır.

Bu ön inceleme göstermektedir ki mevcut yaklaşımlar, özellikle:

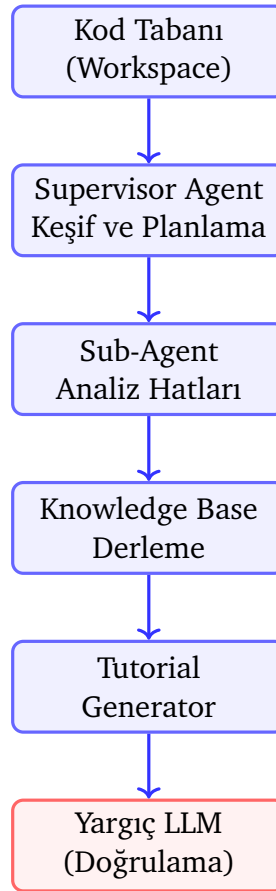
- tek ajanlı sistemlerde bağlam sınırı ve uzun soluklu planlama güçlüğü,
 - yönetici-işçi modellerinde verimlilik ve esneklik kısıtları,
- gibi temel problemleri çözmekte yetersizdir.

Dolayısıyla büyük kod tabanlarında tutarlı, izole ve ölçeklenebilir analiz yapabilmek için Sub-Agent mimarisine dayalı bir çözümün gerekliliği ortaya çıkmaktadır. Her bir Sub-Agent'ın kendi sorumlu dizininde çalışması, çıktılarının güvenli biçimde bir Knowledge Base'de toplanması ve Supervisor Agent'ın bağlam yükünden arındırılması bu yaklaşımın temel motivasyonunu oluşturur.

2 SİSTEM ANALİZİ VE FİZİBİLİTE

2.1 Sistem Analizi

Bu bölüm, modern çok ajanlı yapay zekâ metodolojilerini takip ederek “Otonom Ajan Yaklaşımıyla Kod Veritabanından Eğitim İçeriği Üreten” projenin kapsamını netleştirir. Amaç şekil 2.1’de gösterildiği üzere, kod tabanındaki bilgiyi dağıtık bir ajan mimarisi ile derleyip kalıcı bir Bilgi Bankası üretmek ve bu çıktıyı pedagojik değeri yüksek eğitim materyallerine dönüştürmektir.

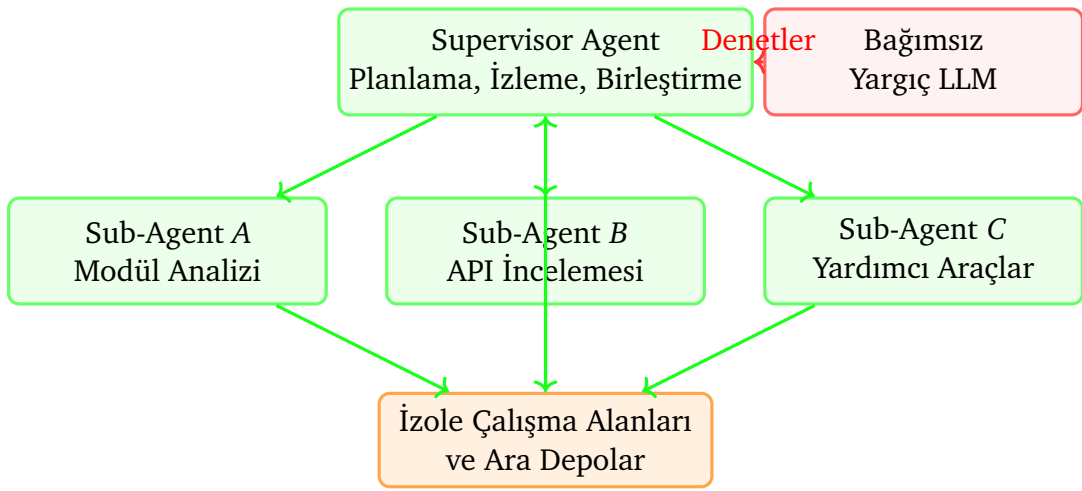


Şekil 2.1 Bilgi üretim hattının üst düzey görünümü.

Mimari, iki aşamalı Sub-Agent kurgusu üzerine kuruludur. Supervisor Agent keşif,

planlama ve orkestrasyon görevlerini yürütür; bağlam yükünü azaltmak için işleri Sub-Agent'lara devreder. Sub-Agent'lar kendi çalışma alanlarında ayrı olarak analiz yapar, bulgularını kalıcı depolara kaydeder ve Supervisor Agent'a raporlar. Supervisor Agent bu çıktıları birleştirerek nihai Knowledge Base'i oluşturur; ardından içerik üretimi için Tutorial Generator bileşenini tetikler.

Deterministik görev bölme yaklaşımıyla kaynak dosya altındaki ilk seviye klasörler ayrı Sub-Agent'lara atanır. Alternatif olarak Supervisor Agent, planlama parametreleri ve bir yapılacaklar listesi üzerinden görevleri dinamik biçimde güncelleyebilir. Sub-Agent'ların kod tabanına yalnızca okuma, kendi çalışma alanlarına ise yazma yetkisi vardır; böylece bağlam yönetimi ve çıktı doğrulaması sadeleşir.



Başarı Kriterleri ve Metrikler. Sistemin başarısı, teorik formüller yerine, sistem bileşenleri tarafından toplanan operasyonel veriler ve Yargıç LLM tarafından gerçekleştirilen kalite ölçümleri ile değerlendirilir.

- **Operasyonel Verimlilik (Efficiency):** Sistemin bir projeyi analiz etme maliyeti şu üç temel metrikle izlenir:
 - *Token Tüketimi:* Toplam Girdi/Çıktı token sayısı (Maliyet analizi için).
 - *Süre (Duration):* Knowledge Base ve Tutorial fazlarının tamamlanma süreleri.
 - *Araç Kullanımı (Tool Calls):* Ajanların dosya okuma ve yazma sıklığı.
- **İçerik Kalitesi (Yargıç LLM):** Üretilen eğitim içeriklerinin kalitesi, bağımsız bir LLM (Yargıç) tarafından 1-5 ölçeğinde puanlanır. Temel kriterler şunlardır:
 - *Doğruluk (Accuracy):* Kod örneklerinin gerçek kaynak kodla uyumu.
 - *Tamamlayıcılık (Completeness):* Konunun ne kadar kapsandığı.

- *Pedagoji*: Anlatım sırasının mantıksal akışı.
- **Süreç Dayanıklılığı (Robustness):**
 - *Bağlam Taşması (Context Overflow)*: LLM'in hafıza sınırını aşma olaylarının sayısı.
 - *Toparlanma Başarısı*: Hata sonrası 'retry' mekanizmasının başarı oranı.

2.2 Gereksinim Analizi

Gereksinimler üç düzeyde sınıflandırılmıştır: kullanıcı rolleri, sistem modülleri ve işlevsel akış. Sistemin iki temel kullanıcısı bulunmaktadır. Birincisi, Bilgi Bankası ve eğitim çıktıları üzerinden kod tabanını öğrenen geliştiricidir. İkincisi ise ajan süreçlerini başlatan, izleyen ve üretilen çıktının bütünlüğünü doğrulayan sistem operatörüdür.

Sistem modülleri dört ana bileşenden oluşur. Supervisor Agent; keşif, planlama, görev dağıtımı ve Sub-Agent'lerden gelen çıktıları birleştirme görevlerini üstlenen merkezî unsurdur. Sub-Agent'lar; deterministik olarak veya koşula bağlı şekilde üretilen, belirli izinleri veya görev kümelerini analiz eden ve izole çalışma alanlarında faaliyet gösteren ajanlardır. Knowledge Base deposu, tüm Sub-Agent özetlerinin toplandığı yapısal bir klasör düzeni sunarken, Tutorial Generator bileşeni bu Knowledge Base'i kullanarak pedagojik değer taşıyan eğitim materyallerini bağımsız bir çıktı klasörü altında üretir.

Sistemin işlevsel akışı ardışık beş aşamada gerçekleşir. İlk aşamada Supervisor Agent kod tabanının dosya yapısını çıkarır, kritik bileşenleri analiz eder ve deterministik ya da dinamik olarak görev listesini oluşturur. İkinci aşamada bu görevler ilgili Sub-Agent'lara dağıtılır. Üçüncü aşamada Sub-Agent'lar, sorumlu oldukları kod parçalarını inceleyerek özetlerini kendi çalışma izinlerine kaydeder. Dördüncü aşamada Supervisor Agent bu alt çıktıları bir araya getirerek bütünlüklü Knowledge Base'i oluşturur ve kalite kontrolünü gerçekleştirir. Son aşamada Tutorial Generator, oluşturulan bu Knowledge Base'i geliştiricilerin kolay öğrenebileceği bir pedagojik formata dönüştürür.

İşlevsel olmayan gereksinimler ise sistemin güvenlik, izlenebilirlik, genişletilebilirlik ve performans özelliklerini belirlemektedir. Sistem, özellikle izin gezintisi (path traversal) zafiyetlerine karşı güvenli olacak şekilde tasarlanmalı; tüm API anahtarları gizli tutulmalı ve kod tabanına gömülmemelidir. Ayrıca ajanların karar verme süreçleri, özellikle görev delegasyonu ve dosya işlemleri Phoenix aracı üzerinden ayrıntılı biçimde loglanmalıdır. Mimari, yeni araçların veya ajan türlerinin

eklenmesine açık bir genişletilebilirlik sunmalıdır. Son olarak performans açısından, bir prototip olması nedeniyle süreç karmaşıklığını azaltmak amacıyla Sub-Agent'ların paralel değil, sıralı olarak çalıştırılması tercih edilmiştir.

2.3 Fizibilite

2.3.1 Teknik Fizibilite

2.3.1.1 Donanım

Model eğitimi hedeflenmediği için yüksek donanım yatırımı gerekmez. LLM çağrıları API tabanlıdır; standart bir geliştirici bilgisayarı yeterlidir. İş yükü Sub-Agent'lara bölündüğü için bellek baskısı artmaz, uzun dosya okumaları engellenir.

2.3.1.2 Yazılım

- **İşletim Sistemi:** Windows, macOS veya Linux.
- **Programlama Dili:** Python 3.10+ (Backend), TypeScript/React (Frontend).
- **Çerçeveler ve Araçlar:** smolagents (Ajan Orkestrasyonu), LiteLLM (AI Gateway), Next.js (Web Arayüzü), Phoenix (İzleme).

2.3.2 Yasal Fizibilite

Kullanılan kütüphaneler MIT veya Apache 2.0 benzeri açık kaynak lisanslarıyla dağıtılır. Analiz edilecek kod tabanının proje ekibine ait olması veya lisans uyumunun sağlanması yeterlidir.

2.3.3 Ekonomik Fizibilite

Projenin ekonomik fizibilitesi, düşük başlangıç maliyeti ve yüksek katma değer üretme potansiyeli sayesinde olumlu olarak değerlendirilmektedir. Sistem, büyük ölçüde açık kaynaklı yazılımlar ve bulut tabanlı LLM servisleri üzerine kurulu olup, merkezi ve sürekli bir altyapı yatırımı gerektirmemektedir.

2.3.3.1 Giderler

Temel bileşenler ücretsizdir. Maliyet kalemleri ağırlıklı olarak mühendislik eforu ve LLM API çağrılarından oluşmaktadır. Geliştirme ve test aşamalarında Google AI Studio veya OpenRouter üzerinden sunulan ücretsiz ya da düşük maliyetli modeller kullanılabilir. Son kullanıcıya sunulan nihai dokümantasyon ve

eğitim içeriklerinin üretiminde ise daha güçlü modeller tercih edilerek kalite–maliyet dengesi sağlanmaktadır.

Başlıca gider kalemleri aşağıda özetlenmiştir:

- **Geliştirme Maliyeti:** Sistem mimarisi, çoklu ajan orkestrasyonu ve prompt mühendisliği için harcanan mühendislik eforu.
- **LLM API Maliyetleri:**
 - *Gemini 2.5 Pro*: Ortalama bir proje analizi için yaklaşık \$0.10 – \$0.50.
 - *GPT-OSS-120B*: Yüksek kalite gerektiren son tutorial üretimi aşamalarında sınırlı ve seçici kullanım.

2.3.3.2 Katma Değer

Sistemin sağladığı katma değer, doğrudan parasal kazançtan ziyade zaman tasarrufu ve verimlilik artışı üzerinden ortaya çıkmaktadır. Bu bağlamda öne çıkan kazanımlar şunlardır:

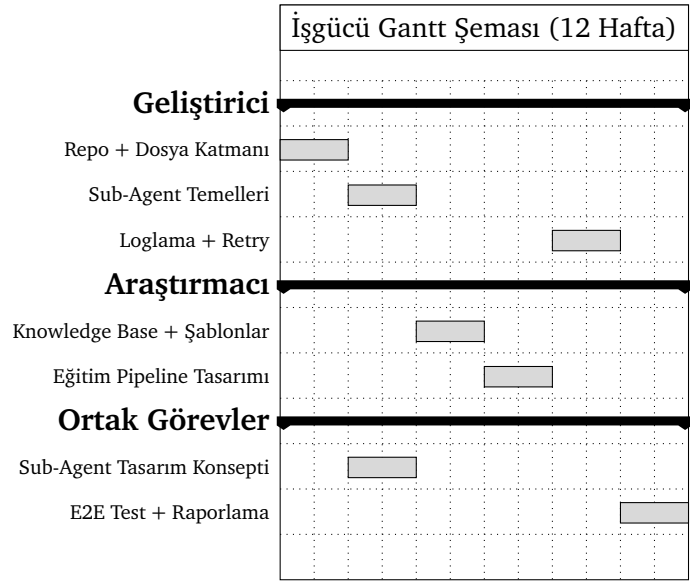
- Dokümantasyon ve proje öğrenme süreçlerinde %80’den fazla zaman tasarrufu.
- Yeni geliştiricilerin projeye adaptasyon süresinin önemli ölçüde kısalması (hızlı onboarding).
- Kod tabanı değiştikçe ajanların yeniden çalıştırılmasıyla sürekli güncel dokümantasyon elde edilmesi.
- İlk kullanımda elde edilen zaman kazancı sayesinde sistemin kendi maliyetini kısa sürede amorti edebilmesi.

Bu yönleriyle proje, hem bireysel geliştiriciler hem de kurumsal yazılım ekipleri için ekonomik açıdan sürdürülebilir ve ölçeklenebilir bir çözüm sunmaktadır.

2.3.4 İşgücü ve Zaman Fizibilitesi

Ekip: 2 kişi (yazılım/araştırma odaklı).

Genel Yaklaşım: İlk sürüm için 8–12 hafta aralığında bir geliştirme süresi öngörülür. Detaylı şema şekil 2.2’de sunulmuştur.



Şekil 2.2 İşgücü Fizibilitesi – Rol Bazlı Gantt Şeması

3 SİSTEM TASARIMI

Bu bölümde, geliştirilen çok ajanlı yapay zekâ analiz platformunun sistem tasarımı detaylandırılmaktadır. Tasarım mimarisi; kullanıcı etkileşimi, veri yönetimi ve nesne yönelimli modelleme olmak üzere üç temel ekseninde kurgulanmıştır. Amaç, karmaşık yazılım projelerini otonom olarak analiz edebilen ve eğitsel içerik üretebilen modüler, güvenli ve ölçeklenebilir bir yapı sunmaktır.

3.1 Arayüz Tasarımı

Sistem, kullanım kolaylığı ve operasyonel esnekliği bir arada sunmak amacıyla ikili bir arayüz yapısı benimsemiştir: Operasyonel süreçler için **Komut Satırı Arayüzü (CLI)** ve analitik çıktılar için **Web Tabanlı Kontrol Paneli**.

3.1.1 Komut Satırı Arayüzü (CLI)

Analiz süreçlerinin başlatılması ve yönetilmesi, geliştiricilerin alışkın olduğu terminal ortamı üzerinden gerçekleştirilir. Kullanıcılar, tek bir komut ile hedef projeyi sisteme tanıtır ve analiz sürecini başlatır. Sistemin CLI katmanı, arka planda çalışan yapay zekâ ajanlarının durumunu, kaynak tüketimini ve ilerleme yüzdelelerini anlık olarak terminal ekranına yansıtır. Bu yalın tasarım, sistemin CI/CD (Sürekli Entegrasyon/Dağıtım) boru hatlarına kolayca entegre edilmesine olanak tanır.

3.1.2 Web Tabanlı Kontrol Paneli (Web Dashboard)

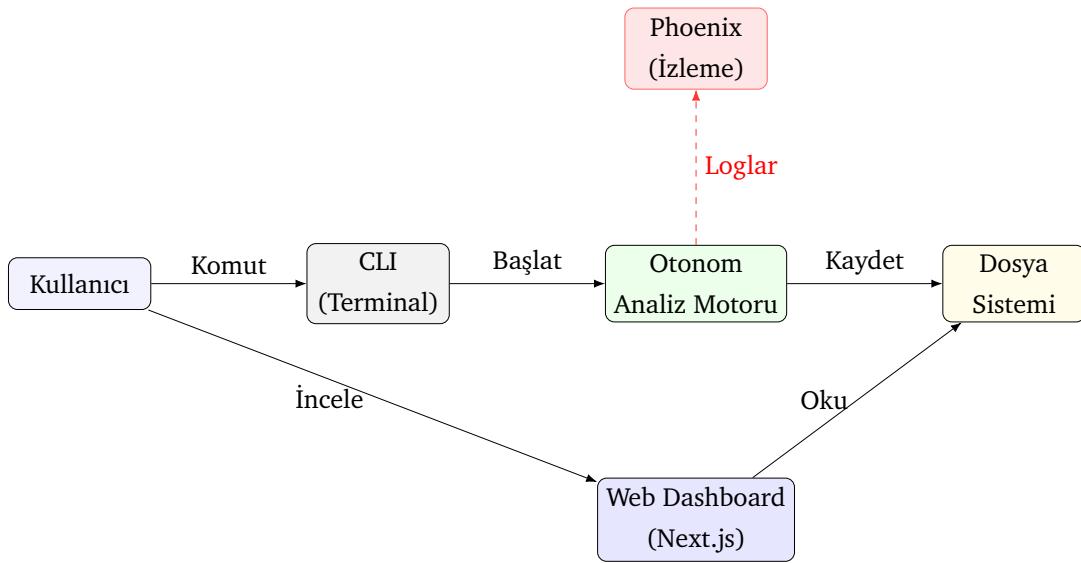
Otonom analiz tamamlandıktan sonra üretilen Knowledge Base ve eğitim materyallerinin son kullanıcıya sunulması için modern web teknolojileri (Next.js ve React) kullanılarak geliştirilmiş bir arayüz mevcuttur. Bu panelin temel işlevleri şunlardır:

- **Proje Kütüphanesi:** Analiz edilen farklı kod tabanlarının görsel kartlar

halinde listelenmesi ve yönetilmesi.

- ****Etkileşimli İçerik Görüntüleyici:**** Ajanlar tarafından üretilen Markdown formatındaki ders içeriklerinin, kod renklendirme (syntax highlighting) ve dinamik diyagram destekli olarak okunabilir hale getirilmesi.
- ****Akıllı Gezinme:**** Oluşturulan müfredat içerikleri arasında hiyerarşik geçiş imkanı.

Şekil 3.1’te sistemin operasyonel ve analitik arayüzleri arasındaki veri akış diyagramı sunulmuştur.



Şekil 3.1 Kullanıcı etkileşimi ve veri akış şeması.

3.2 Veri Yapısı Tasarımı

Proje mimarisinde, karmaşık veritabanı yönetim sistemleri yerine, taşınabilirlik ve şeffaflığı ön planda tutan, dosya sistemi tabanlı (filesystem-based) bir veri yaklaşımı tercih edilmiştir. Bu tasarım kararı, üretilen dokümantasyonun versiyon kontrol sistemleri (örn. Git) ile uyumlu olmasını ve projenin kendisiyle birlikte saklanabilmesini sağlar.

3.2.1 Yapılandırma Yönetimi

Sistemin davranış kalıpları ve parametreleri, merkezi yapılandırma dosyaları üzerinden yönetilir. Model seçimleri, API erişim anahtarları ve global sistem ayarları tek bir noktadan kontrol edilirken, web arayüzünde görüntülenecek metadata bilgileri ayrıştırılmış formatlarda saklanır.

3.2.2 İzole Çalışma Alanları (Scoped Filesystem)

Veri güvenliğini sağlamak ve projeler arası çakışmaları önlemek adına "İzole Dosya Sistemi" mimarisi uygulanmıştır. Her analiz süreci, kendine tahsis edilen üç katmanlı bir izin yapısı içinde çalışır:

1. **Kaynak Katmanı (Okuma):** Ajanların sadece okuma iznine sahip olduğu, orijinal kod tabanının bulunduğu güvenli alan.
2. **Bellek Katmanı (Geçici):** Analiz sürecinde oluşturulan ara özetlerin ve notların tutulduğu çalışma alanı.
3. **Çıktı Katmanı (Yazma):** Nihai eğitim materyallerinin ve raporların derlendiği sonuç klasörü.

Bu yapı sayesinde, yapay zekâ ajanlarının proje dosyalarına zarar vermesi engellenmiş ve veri bütünlüğü garanti altına alınmıştır.

3.3 Nesneye Yönelik Tasarım ve Ajan Mimarisi

Sistemin yazılım mimarisi, klasik statik sınıf hiyerarşileri yerine, esnekliği maksimize eden **Bileşimsel (Composition-based)** ve **Dinamik** bir tasarım modeline dayanır. Ajanlar, sabit kod blokları olarak değil, çalışma zamanında (runtime) ihtiyaca göre bir araya getirilen yetenek kümeleri olarak kurgulanmıştır.

3.3.1 Dinamik Ajan Yapısı

Platform içerisindeki her bir ajan (Supervisor Agent, Sub-Agent, Tutorial Generator), üç temel bileşenin dinamik birleşimiyle oluşturulur:

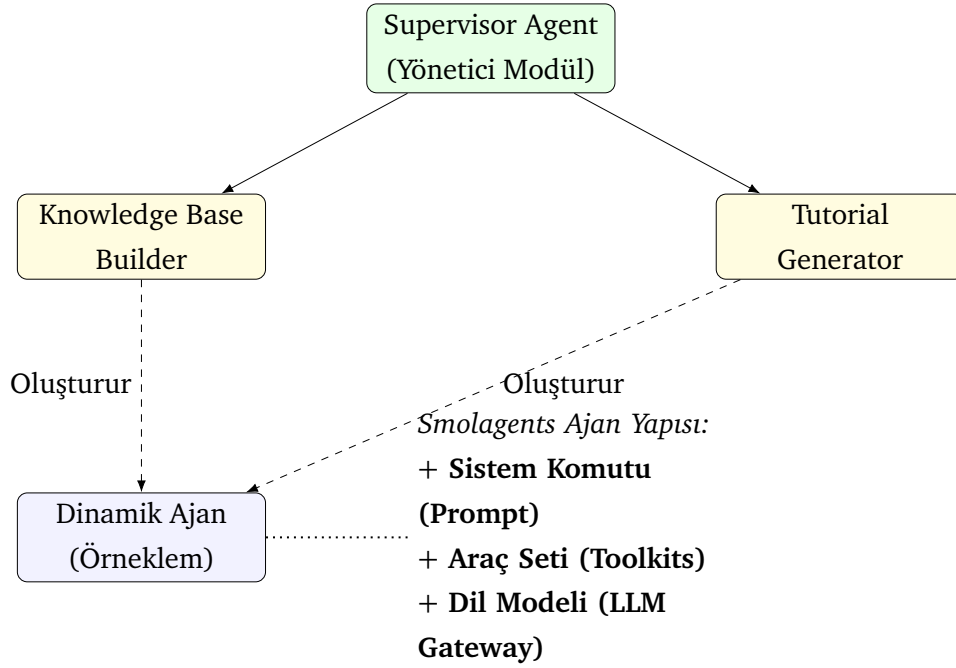
- **Kimlik ve Rol Tanımı (Prompt):** Ajanın amacını, kurallarını ve davranış biçimini belirleyen doğal dil talimatlarıdır.
- **Yetenek Seti (Toolkit):** Ajanın dosya okuma, komut çalıştırma veya internette arama yapma gibi teknik becerilerini tanımlayan modüler fonksiyon kütüphaneleridir.
- **Dil Modeli (LLM):** Görevin karmaşıklığına göre seçilen, ajana zeka sağlayan işlem birimidir.

Bu modüler yapı, sisteme yeni yeteneklerin eklenmesini veya ajan davranışlarının değiştirilmesini, kaynak kodda köklü değişiklikler yapmadan mümkün kılar.

3.3.2 Orkestrasyon Katmanı

Ajanların koordinasyonu ve görev dağılımı, üst seviye yönetici sınıflar tarafından sağlanır. Sistemde ana yönetici rolünü üstlenen bu yapı, analiz sürecini mantıksal aşamalara (Bilgi Toplama, Planlama, İçerik Üretimi) böler ve her aşama için gerekli ajanları senkronize bir şekilde devreye sokar.

Şekil 3.2’de sistemin dinamik nesne mimarisi özetlenmiştir.



Şekil 3.2 Orkestrasyon yapısı ve dinamik ajan üretimi.

Bu mimari tercih, sistemin bakım maliyetini düşürmekte ve farklı programlama dilleri veya teknolojiler için kolayca özelleştirilebilmesine olanak tanımaktadır.

4.1 Uygulama Tanıtımı

Geliştirilen sistem, son kullanıcıların karmaşık komut dizileriyle uğraşmadan, doğrudan doğal dil ve basit terminal konfigürasyonlarıyla çalıştırabileceği bir yapı sunmaktadır. Uygulama, ana dizinde bulunan çalıştırıcı betik üzerinden yönetilmektedir. Bu betik, gerekli ortam değişkenlerini yükler, sanal ortamı kontrol eder ve Python tabanlı komut satırı arayüzünü başlatır.

Kullanıcılar için iki temel çalışma modu tasarlanmıştır:

1. **Deep Agent Modu:** Sub-Agent mimarisinin tam kapasiteyle çalıştığı, önce Knowledge Base oluşturup ardından eğitim içeriği üreten mod.
2. **Baseline Modu:** Karşılaştırma amacıyla geliştirilen, Sub-Agent ve Knowledge Base kullanmadan doğrudan tek bir ajanın (ReAct mimarisi) tüm işi yapmaya çalıştığı mod.

Sistem, otonom analiz ve içerik üretimi için yapılandırılmış komut setleri ile çalıştırılmaktadır. Kullanıcı, hedef kod tabanının yolunu belirterek analizi başlatabilir; karşılaştırma amaçlı olarak sistemin alt-ajan kullanmadan doğrudan tekil bir yapı üzerinden ilerlediği bir temel (baseline) modu da desteklenmektedir.

Uygulama çalıştığında, arka planda olay tabanlı bir loglama sistemi devreye girer. Kullanıcı terminalde anlık ilerlemeyi (ajanın o an hangi dosyayı okuduğu, hangi kararı verdiği) görebilirken, üretilen Markdown formatındaki eğitim dokümanları ve analiz raporları ilgili dizin altında otomatik olarak oluşturulur.

4.2 Test Sonuçları

Sistemin başarımı, *Sistem Analizi* bölümünde belirlenen gereksinimler doğrultusunda, özellikle maliyet etkinliği ve bağlam (context) yönetimi açısından test edilmiştir.

Bu testlerde "Agent Lightning" kütüphanesi örnek proje olarak kullanılmış ve aynı donanım/ortam şartlarında hem Baseline hem de Deep Agent modları çalıştırılmıştır.

Elde edilen veriler, sistemin işlem yükü, token tüketimi ve araç kullanım sıklığı gibi performans metriklerini ortaya koymaktadır.

4.2.1 Maliyet ve Bağlam Analizi (Deney 4)

Büyük Dil Modelleri ile çalışan sistemlerde en kritik darboğaz, modelin bağlam penceresi ve token başına maliyettir. Aşağıdaki Tablo 4.1, Baseline ve Deep Agent yaklaşımlarının "Tutorial Üretim Fazı" için karşılaştırmalı performansını göstermektedir.

Tablo 4.1 Baseline ve Deep Agent Modellerinin Tutorial Üretim Fazı Karşılaştırması

Metrik	Baseline Agent	Deep Agent
Toplam Süre (saniye)	378.25	620.98
Girdi Token Sayısı (Input)	1,732,968	3,236,280
Çıktı Token Sayısı (Output)	77,745	145,053
Toplam Token Maliyeti	1.81 Milyon	3.38 Milyon
Araç Çağrı Sayısı (Tool Calls)	32	43
Bağlam Taşması (Overflow)	0	0

Analiz: Tablo incelendiğinde, Deep Agent mimarisinin Baseline'a göre yaklaşık iki kat daha fazla token tükettiği ve işlemin daha uzun sürdüğü görülmektedir. İlk bakışta bu bir dezavantaj gibi görünse de, çıktının kalitesi ve derinliği göz önüne alındığında (Bkz. Bölüm 4.3), bu "fazladan" maliyetin sisteme **bağlam zenginliği** olarak geri döndüğü anlaşılmaktadır. Baseline ajan, daha az token harcamasına rağmen yüzeysel ve genel geçer çıktılar üretirken; Deep Agent, oluşturduğu Knowledge Base'i bağlamına yüklediği için (input token artışı buradan kaynaklanmaktadır) çok daha detaylı, kod referanslı ve teknik doğruluğu yüksek içerikler üretmiştir.

Ayrıca, her iki sistemde de "Bağlam Taşması (Context Overflow)" olayının "0" olması, geliştirilen hız sınırlamalı dil modeli sarmalayıcısının ve token yönetim mekanizmasının başarılı bir şekilde çalıştığını kanıtlamaktadır.

4.3 Doğrulama

Sistemin ürettiği çıktıların kalitesi ve kullanıcı gereksinimlerini karşılama düzeyi, güncel bir yöntem olan "Judge LLM" (Yargıç LLM) metodu ile doğrulanmıştır.

4.3.1 Yargıç LLM Değerlendirmesi (Deney 2)

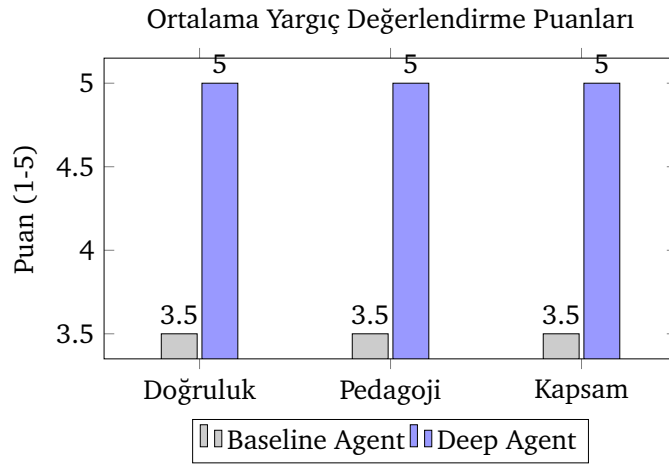
Bu deneyde, SOTA (State-of-the-Art) kabul edilen **Claude 4.5 Sonnet** ve **GPT-OSS-120B** modelleri "tarafsız yargıç" olarak atanmıştır. Yargıçlara, hem Baseline (Seri A) hem de Deep Agent (Seri B) tarafından üretilen eğitim dokümanları, kaynak kod ile birlikte verilmiş ve şu üç kriterde puanlama yapmaları istenmiştir:

1. **Doğruluk (Fidelity):** Üretilen kod örnekleri ve açıklamalar gerçek kod tabanıyla örtüşüyor mu? Halüsinasyon var mı?
2. **Pedagoji (Pedagogy):** Anlatım, bir öğrencinin anlayabileceği şekilde basitten zora doğru mu? Örnekler öğretici mi?
3. **Kapsam (Coverage):** Kod tabanının kritik özelliklerine değinilmiş mi yoksa sadece yüzeyde mi kalınmış?

Yargıçların değerlendirme sonuçları Tablo 4.2'de özetlenmiştir.

Tablo 4.2 Yargıç LLM'lerin Kalite Değerlendirmesi (5 üzerinden puan)

Yargıç Model	Doğruluk		Pedagoji		Kapsam	
	Base	Deep	Base	Deep	Base	Deep
Claude 4.5 Sonnet	3	5	3	5	3	5
GPT-OSS-120B	4	5	4	5	4	5



Şekil 4.1 Baseline ve Deep Agent modellerinin kalite metrikleri bazında karşılaştırılması.

Sonuç Yorumu: Her iki yargıç modeli de ****Deep Agent (Seri B)**** tarafından üretilen dokümanları "açık ara kazanan" (Winner: B) olarak ilan etmiştir.

- **Doğruluk:** Baseline modelin, kullanılmayan (deprecated) API'leri önerdiği veya sınıf isimlerini yanlış yazdığı tespit edilmiştir. Deep Agent ise Knowledge Base'den doğrulama yaptığı için %100'e yakın doğruluk sağlamıştır.
- **Pedagoji:** Deep Agent; "Hello World" örneğinden başlayıp, ileri seviye konulara (Distributed Training, Async Execution) doğru giden yapılandırılmış bir müfredat sunmuştur. Baseline ise konuları kopuk parçalar halinde sunmuştur.

4.3.2 Gereksinim Uyumluluk Analizi

Projenin başlangıcında (Bkz. Bölüm 2.2) belirlenen işlevsel ve işlevsel olmayan gereksinimlerin karşılanma durumu yapılan son denetimlerle (audit) doğrulanmıştır.

- **Sub-Agent Mimarisi:** ✓Tamamlandı. Supervisor Agent dinamik olarak Sub-Agent'lar oluşturabilmekte ve izole çalışma alanlarını yönetebilmektedir.
- **Güvenlik (Path Traversal):** ✓Tamamlandı. Geliştirilen kısıtlanmış dosya erişim modülü, ajanların sadece izin verilen dizinlere erişmesini garanti altına almıştır.
- **Knowledge Base Üretimi:** ✓Tamamlandı. Kod tabanından otomatik olarak kapsamlı bir Markdown Knowledge Base oluşturulabilmektedir.
- **Raporlama ve İzleme:** ✓Tamamlandı. Hem terminal çıktıları hem de JSON formatındaki metrik dosyaları ile süreç şeffaf hale getirilmiştir.

Bu sonuçlar, geliştirilen sistemin hedeflenen otonom eğitim içerik üretimi görevini başarıyla yerine getirdiğini ve Deep Agent mimarisinin, geleneksel yöntemlere göre kalite açısından belirgin bir üstünlük sağladığını doğrulamaktadır.

5

SONUÇ VE TARTIŞMA

Bu çalışmada, büyük ve karmaşık kod tabanlarını otomatize etmek amacıyla "Sub-Agent" mimarisine dayalı otonom bir yapay zekâ sistemi tasarlanmış ve prototipi geliştirilmiştir. Geliştirilen sistem, "Agent Lightning" gibi modern ve dokümantasyonu sınırlı kütüphaneler üzerinde test edilmiş; elde edilen sonuçlar hem nicel metrikler hem de nitel Yargıç LLM değerlendirmeleriyle analiz edilmiştir.

Proje kapsamında elde edilen temel sonuçlar ve tartışma başlıkları aşağıda sunulmuştur:

5.1 Elde Edilen Kazanımlar

- Bağlam Sınırının Aşılması:** Geleneksel tek ajanlı sistemlerin en büyük problemi olan "bağlam penceresi" (context window) kısıtı, geliştirdiğimiz iki aşamalı (Knowledge Base Üretimi → İçerik Üretimi) mimari sayesinde aşılmıştır. Supervisor Agent'ın tüm dosyaları okumak yerine, görevi alt ajanlara dağıtması ve sadece özet bilgiyi (Knowledge Base) kullanması, sistemin binlerce dosyalık projelerde bile "bağlam taşması" yaşamadan çalışmasını sağlamıştır.
- Kalite ve Derinlik Artışı:** Yapılan karşılaştırmalı testler (Bölüm 4.2), Deep Agent stratejisinin token maliyetini artırmasına rağmen (Baseline: 1.8M vs Deep: 3.3M), çıktı kalitesinde tartışılmaz bir üstünlük sağladığını göstermiştir. Baseline ajan yüzeysel bilgiler verirken; Deep Agent, kodun derinliklerindeki mimari desenleri, asenkron çalışma yapılarını ve doğru sınıf isimlerini tespit ederek "kullanılabilir ve doğru" eğitim materyali üretmeyi başarmıştır. Yargıç LLM'ler, Deep Agent'a doğruluk ve pedagoji kriterlerinde tam puan (5/5) vermiştir.
- Güvenli ve İzole Çalışma:** Tasarlanan kısıtlanmış dosya erişim modeli sayesinde, otonom ajanların sadece kendilerine atanan dizinlerde çalışması sağlanmış, böylece olası veri kaybı veya dizin gezintisi (path traversal) zafiyetleri

mimari düzeyde engellenmiştir.

5.2 Kısıtlar ve Maliyet

Sistemin sunduğu yüksek kalitenin bir bedeli olarak işlem süresi ve maliyeti artış göstermiştir. Deep Agent modunda bir kütüphanenin analizi ve tutorial üretimi ortalama 10-20 dakika sürmektedir. Bu süre, bir insanın haftalar süren öğrenme sürecine göre ihmal edilebilir olsa da, anlık yanıt bekleyen interaktif senaryolar için (örneğin IDE içi kod tamamlama) bu mimari şu an için yavaş kalabilir. Ayrıca token tüketiminin yüksek olması, GPT-OSS-120B gibi pahalı modellerle çalışıldığında maliyetli olabilir; bu nedenle sistemin Gemini 2.5 Pro veya açık kaynaklı (OSS) modellerle optimize edilmesi önerilmektedir.

5.3 Gelecek Çalışmalar

Mevcut prototipin başarısı, bu alanda yapılacak ileri çalışmalar için güçlü bir temel oluşturmaktadır. Gelecek dönemde şu geliştirmelerin yapılması hedeflenmektedir:

- **İnsan Deneyleri:** Şu an Yargıç LLM'lerle yapılan doğrulamanın, gerçek yazılım geliştiriciler ve üniversite öğrencileriyle yapılacak A/B testleriyle (Deney 1 ve Deney 3) desteklenmesi.
- **Ablasyon Çalışmaları:** Bilgi Bankası bileşeninin etkisini daha net ölçmek için, sistemin farklı modüllerinin kapatılarak (ör. sadece RAG ile, Knowledge Base olmadan) test edilmesi (Deney 5).
- **Web Arayüzü:** Mevcut CLI yapısının, daha geniş kitlelere hitap edecek, sürükle-bırak özellikli bir Web UI ile zenginleştirilmesi.

Sonuç olarak, bu proje ile otonom ajanların sadece kod yazmakla kalmayıp, yazılan kodu "anlatma ve öğretme" konusunda da insan geliştiricilere güçlü bir asistan olabileceği kanıtlanmıştır.

BİRİNCİ ÜYE

İsim-Soyisim: BORA İLCİ

Doğum Tarihi ve Yeri: 04.10.2003, Bursa

E-mail: bora.ilci@std.yildiz.edu.tr

Telefon: 0541 200 31 04

Staj Tecrübeleri: Orion Innovation Türkiye Ar-Ge Departmanı

İKİNCİ ÜYE

İsim-Soyisim: KAAN EMRE KARA

Doğum Tarihi ve Yeri: 24.09.2002, İstanbul

E-mail: emre.kara@std.yildiz.edu.tr

Telefon: 0530 776 8079

Staj Tecrübeleri: Nef Solution Yazılım Departmanı

Proje Sistem Bilgileri

Sistem ve Yazılım: Windows/macOS/Linux, Python 3.10+, Smolagents

Gerekli RAM: 8 GB

Gerekli Disk: 1 GB